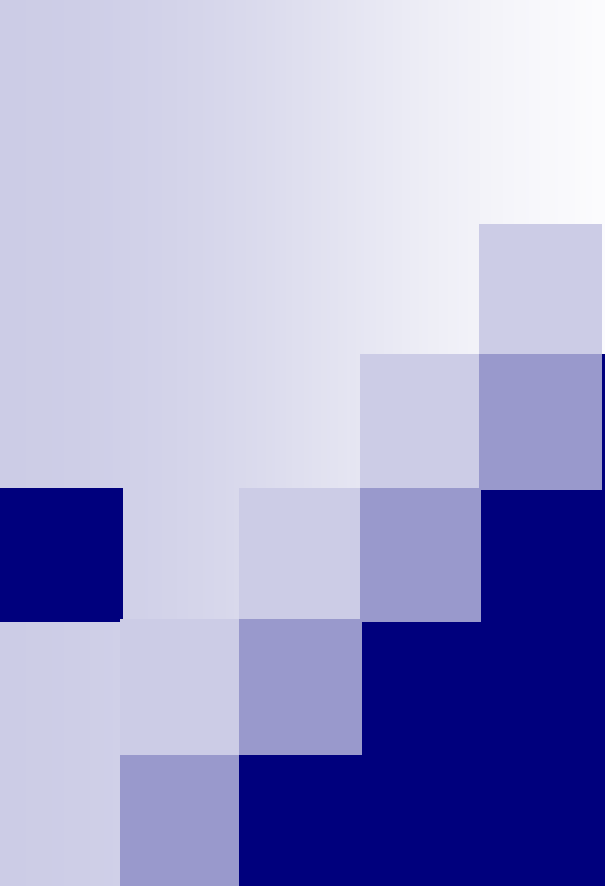




Sexto Congreso Virtual de Microcontroladores.

**Implementación mínima de RISC-V para
sistemas embebidos educativos**

Juan Alarcón

6 de octubre de 2026

Agenda

- Motivación
- Alcance
- Selección de Arquitectura
- Hardware de pruebas
- Primera implementación de núcleo RISC-V
- Segunda implementación de núcleo RISC-V
 - Diferencias entre implementaciones.
 - Datapath y unidad de control.
 - Ejemplo de ejecución
 - Análisis de la ejecución y compilación

Motivación

■ Problema educativo

- Los procesadores “reales” son complejos. Set de instrucciones grandes, documentación poco clara en mucho casos, micro arquitectura, varias etapas de pipeline.
- Hay muchos procesadores “académicos” muy simples pero con set de instrucciones inventados y no se puede relacionar con herramientas estándar como GCC
- Se buscó que puede implementarse en plataformas de bajo costo.
- Sé buscó un hardware sencillo con herramientas estándar.

Alcance. Simplicidad para aprender

- Un procesador tan simple como sea posible
- Implementación en plataformas de bajo costo
- Implementación en HDL plano (en este caso VHDL)
- La prioridad es comprender y modificar el circuito

Selección de la arquitectura

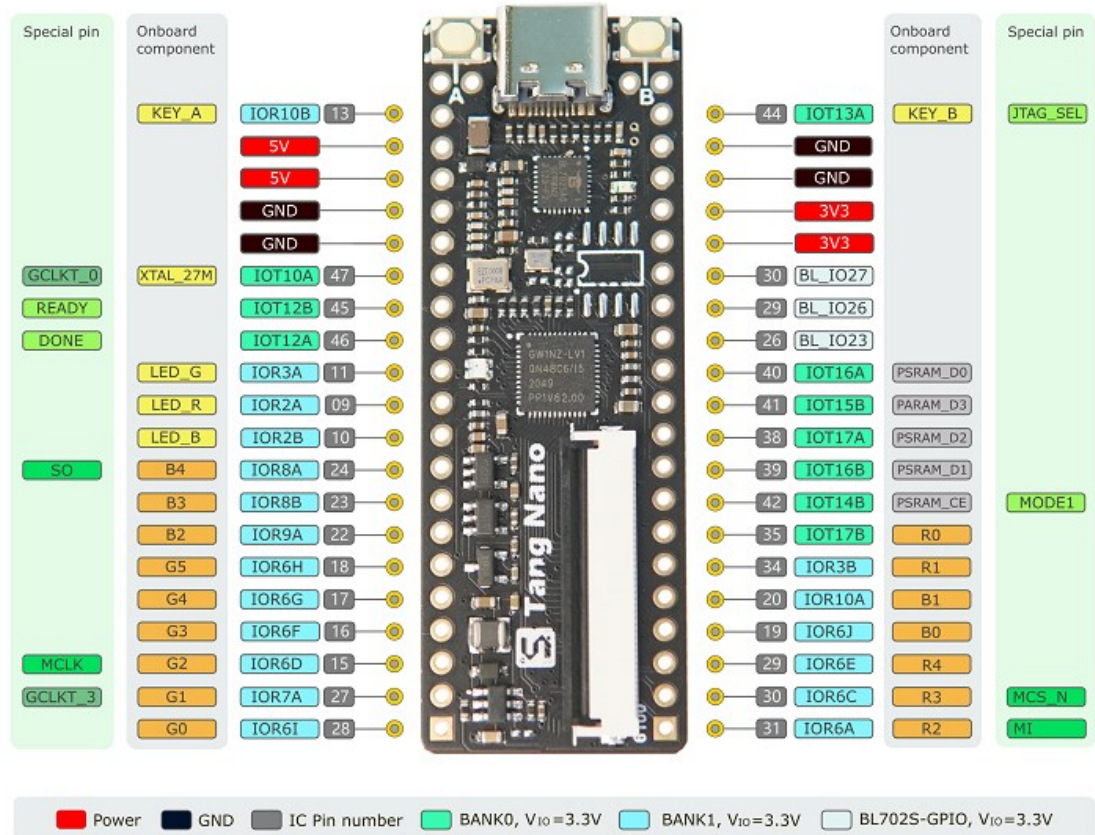
- Complejidad.
 - El RISC-V RV32I tiene 47 instrucciones en total.
- Estructura de las instrucciones.
 - RISC-V RV32I tiene instrucciones regulares todas de 32 bits.
- Documentación
 - RISC-V tiene gran cantidad de documentación y hay mucho material gratuito sobre el tema
- Herramientas de software
 - `risc-v-gnu-toolchain` libre y estándar

Contras de usar RV32I

- Falta de operaciones de multiplicación y división para este set de instrucciones
- Nula compresión de código
- Herramientas de depuración no estándares
- No es el estándar de la industria (ARM es por mucho es estándar y en este caso se tendría que comparar con un Cortex-M0.

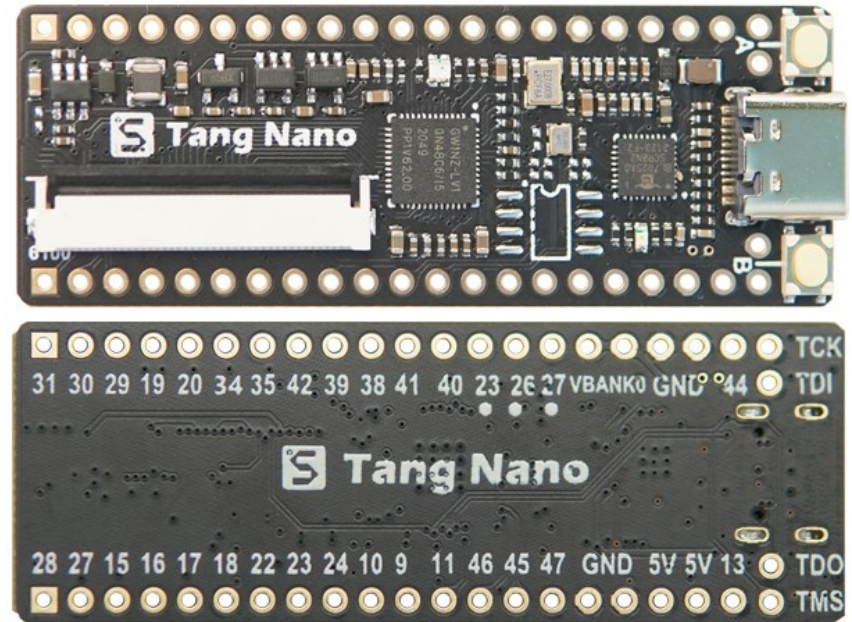
Plataforma de pruebas

- La placa **Tang Nano 1K** está diseñada con la FPGA de la marca Gowin GW1NZ-LV1
- La placa de desarrollo cuenta con un led RGB, dos pulsadores y un oscilador de 27 MHz.
- La placa cuenta con una interfaz para programar tanto la memoria volátil de la FPGA cómo la no volátil
- El esquemático se obtiene en https://dl.sipeed.com/shareURL/TANG/Nano%201K/2_Schematic

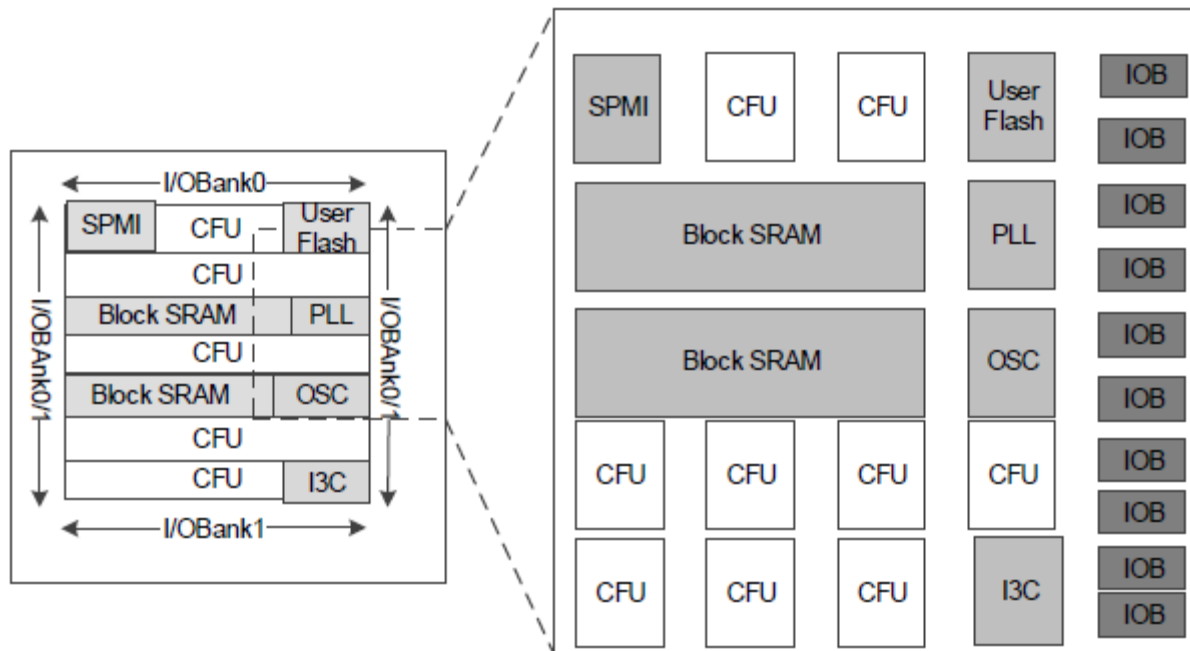


Placa de desarrollo

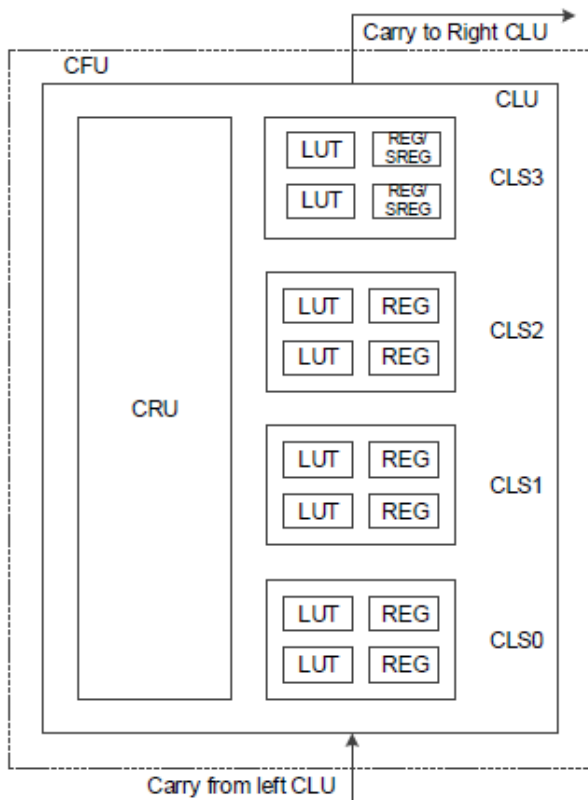
item	Specs
chip	GW1NZ-LV1 FPGA
logic units(LUT4)	1152
Register(FF)	864
Block SRAM(bits)	72K
Number of B-SRAM	4
User flash(bits)	64K
Number of PII	1
Number of I/O Bank	2



FPGA GW1NZ-1

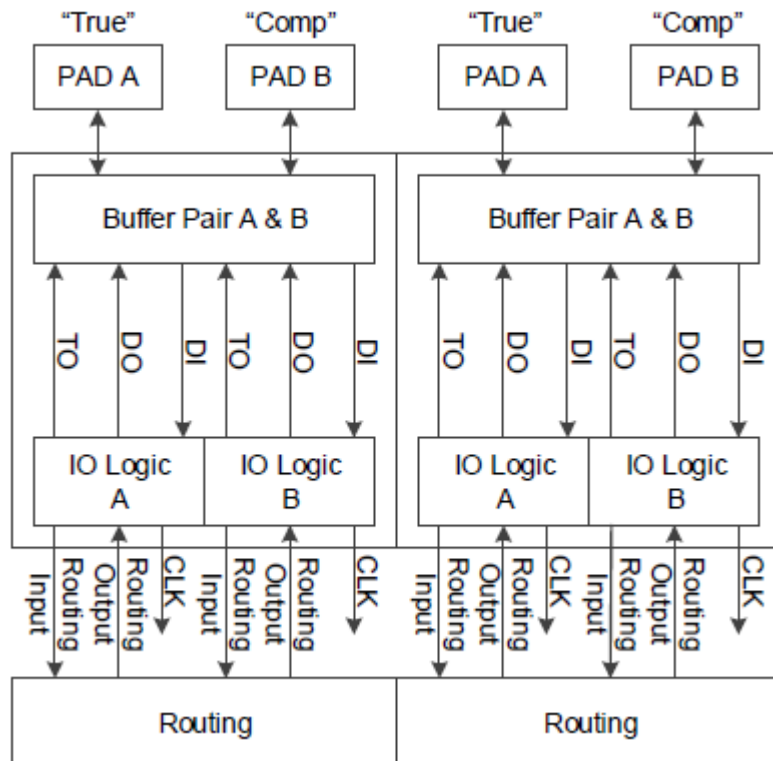


CFU (Configurable Function Unit)



- Cada CFU tiene:
 - Cuatro CLS (configurable Logic Slice).
 - Cada CLS contiene 2 LUT4 y los CLS0, CLS1 y CLS2 tienen dos registros cada uno.
 - CLS3 Se utiliza de manera combinacional principalmente.
- CRU (Clock resource unit) es la parte del bloque que se encarga de los recursos de reloj.
- CLU es la variante de configuración de un CFU puramente combinacional (Configurable Logic Unit)

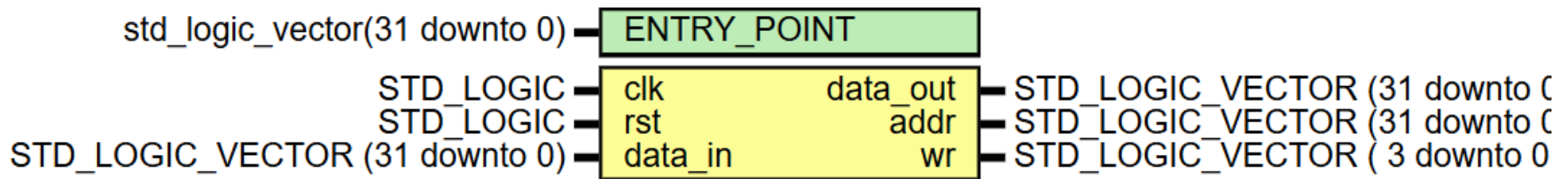
Bloques de entrada salida (IOB)



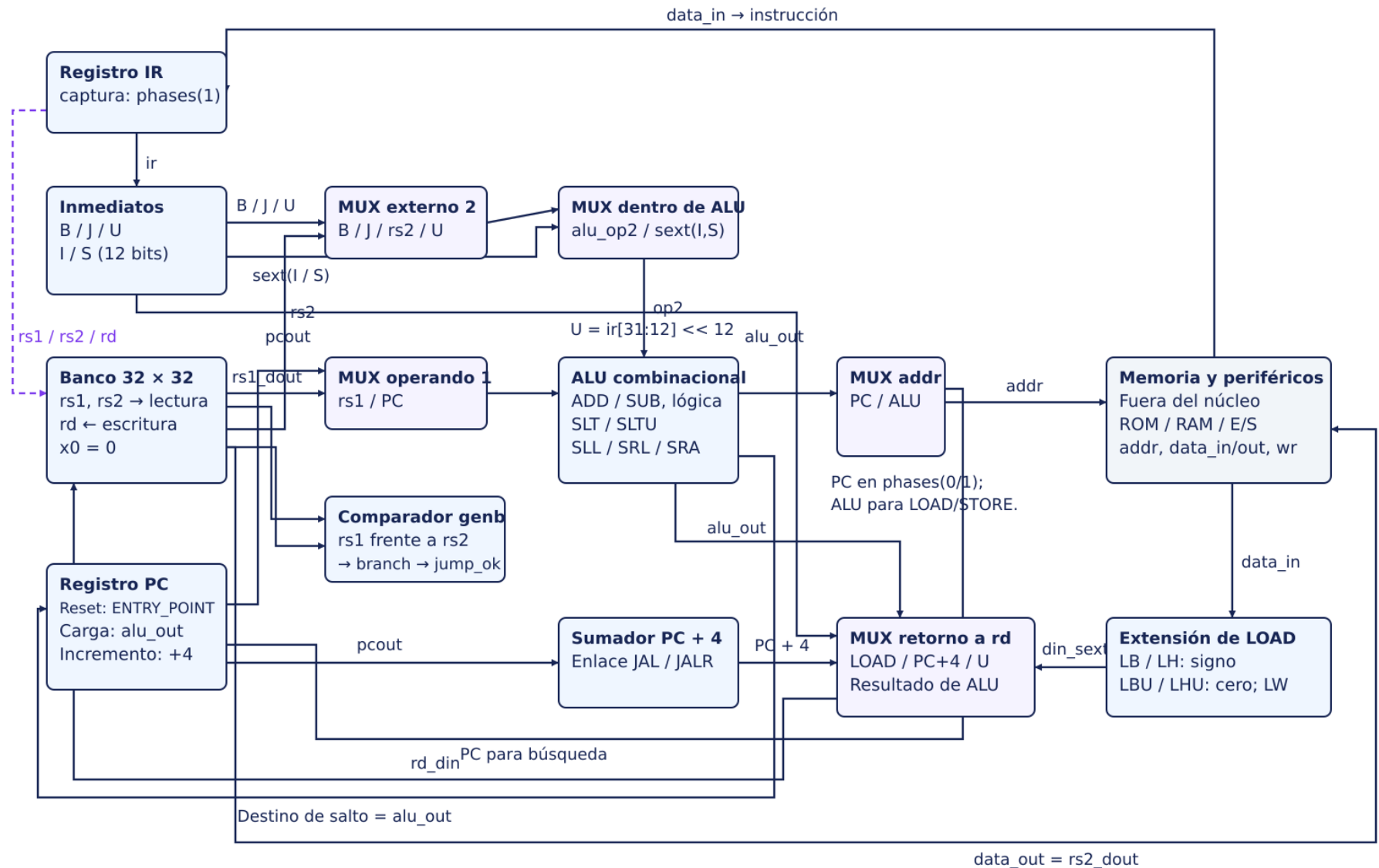
- Cada banco soporta su propia tensión de alimentación (dos bancos).
- Soporta diferentes niveles de tensiones: LVCMOS, LVTTL, etc.
- Corriente de salida configurable
- Slew Rate configurable
- Bus Keeper, Weak Pull-up, Weak Pull-down y Open Drain

Primer intento. jfive

- Primer intento funcional.
- El camino de datos está basado en elementos comunes del diseño digital
- Totalmente sincrónico
- La unidad de control es un contador en anillo de 5 bits
- Se implementó con arquitectura Von Neumann



Datapath



Unidad de control.

- En este caso la unidad de control es un contador en anillo de 5 bits donde cada uno de los bits se usa de la siguiente manera:
 - ☐ 0. Búsqueda de instrucción.
 - ☐ 1. Registro del IR
 - ☐ 2. Decodificación
 - ☐ 3. Espera
 - ☐ 4. Actualización de registros

Resultados (jfive)

Resource	Usage
Logic	759(664 LUT, 95 ALU) / 1152
Register	88 / 957
--Register as Latch	0 / 957
--Register as FF	88 / 957
BSRAM	2 / 4

- Con una frecuencia máxima de 34.279(MHz).
- Faltaba mejorar el software y agregar más capacidades (temporizadores, RAM, etc.).

Programa de ejemplo para hacer
un efecto con 4 leds.
Escribo en la posición de
memoria 0x1000 (puerto de salida)
Considero una Fclk = 12MHz y cada
instrucción necesita 5 ciclos de reloj.

Inicio:

```
addi t0,zero,1  
addi s0,zero,16
```

Actualizar:

```
lui t1,1  
addi t1,t1,0  
sb t0,(t1)
```

Demora:

```
lui t2,300  
#addi t2,zero,4
```

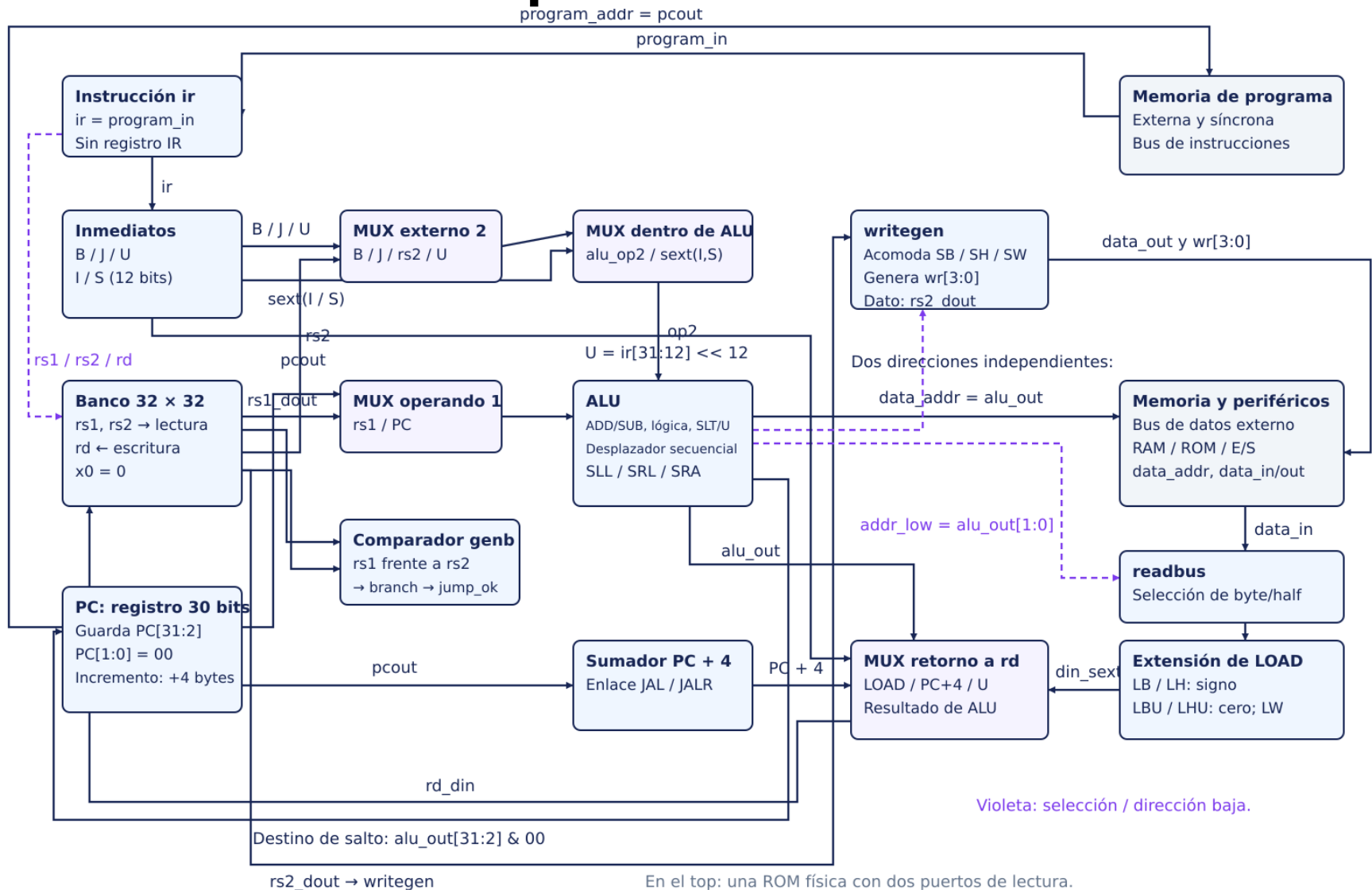
Lazo:

```
addi t2,t2,-1  
bne t2,zero, Lazo  
slli t0,t0,1  
bne s0,t0 Actualizar  
addi t0,zero,1  
jal zero,Actualizar
```

De jfive al riscv32i

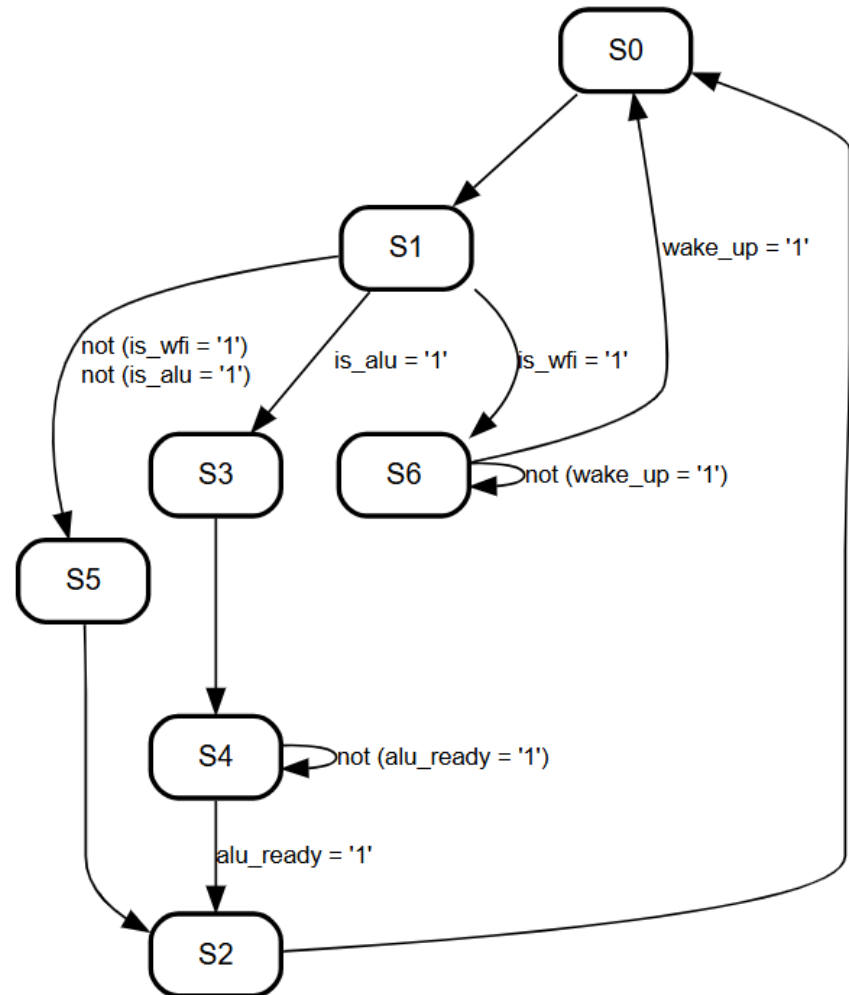
	jfive original	Riscv32i (segunda implementación)
Control	5 fases; contador en anillo	FSM con espera y operaciones secuenciales
Memoria	Bus único; IR registrado	Harvard; Bus de datos y bus de memoria de programa (doble puerto)
Desplazamiento	Combinacional	Un bit por ciclo
Software	GCC y ejemplo de E/S	GCC, arranque, linker script y ejemplo con leds

Datapath riscv32i



Unidad de control

- Máquina de estados de 7 estados.
 - Al hacer desplazamientos secuenciales se agregan ciclos para esperar el fin de la operación.
 - Se separan las operaciones de ALU de las de escritura / lectura.
 - Se agrega ciclo para detener el procesador



Unidad de control (2)

- S0. Búsqueda
- S1. Decodificación
- S5. Espera normal
- S2. Escritura
- S3. ALU modo secuencial
- S4. Espera ALU secuencial
- S6. Procesador en espera

Operaciones de la ALU

cmd	Operación	Bloque
000	ADD / SUB	addsub
001	SLL	desplazamientos
010 / 011	SLT / SLTU	cmp
100	XOR	Lógica combinacional
101	SRL / SRA	desplazamientos
110 / 111	OR / AND	Lógica combinacional

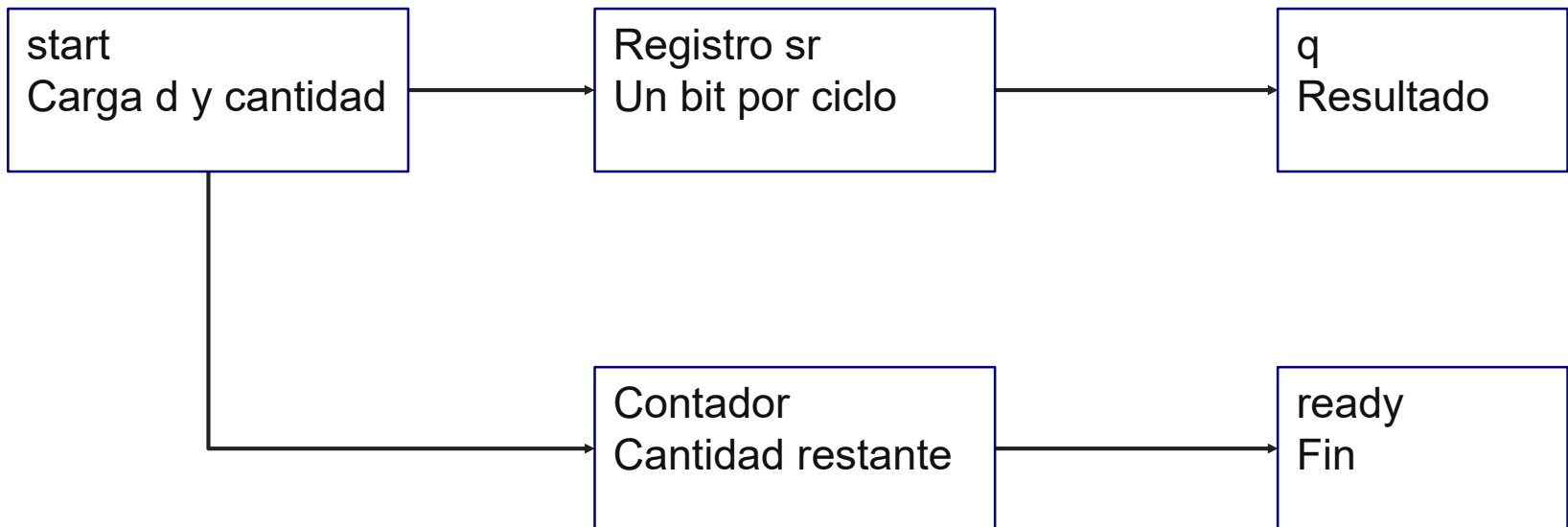
```
as: entity work.addsub(Behavioral)
    generic map(N => N)
    port map( ans => add_param,
              rs1 => rs1,
              rs2 => op2,
              rd => rd_add_sub);
```

```
slt: entity work.cmp(Behavioral)
    generic map(N => N)
    port map (rs1 => rs1,
              rs2 => op2,
              uns => cmd(0),
              eq => open,
              lt => rd_slt);
```

```
srd1: entity work.desplazamientos(Behavioral)
    port map ( clk => clk,
              load => start_des,
              r_n1 => cmd(2),
              sr_in => srla_msb,
              d => rs1,
              r => op2(4 downto 0),
              q => rd_des,
              ready => ready_des);
```

```
with cmd select
    rd <= rd_add_sub
        rd_des
        (0 => rd_slt, others => '0')
        (0 => rd_slt, others => '0')
        rs1 xor op2
        rd_des
        rs1 or op2
        rs1 and op2
        (others => '0')
        when "000", --ADD/SUB
        when "001", --SLL
        when "010", --SLT
        when "011", --SLTU
        when "100", --XOR
        when "101", --SRL/SRA
        when "110", --OR
        when "111", --AND
        when others;
```

Desplazamientos secuenciales

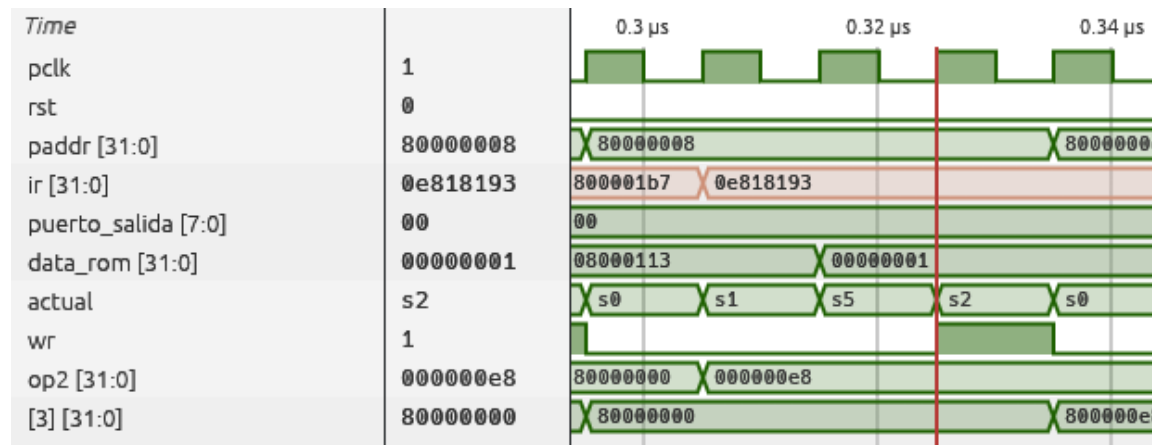


- Los desplazamientos se hacen siempre a la izquierda y se utiliza bit reversal para hacerlos a derecha.
 - $\text{ShiftRight}(D, N) = \text{Reverse}(\text{ShiftLeft}(\text{Reverse}(D), N))$

Modelo de programa

- Implementa los 32 registros de x0 a x31 donde x0 siempre se lee como cero.
- Contador de programa que indica la instrucción a buscar. En esta implementación los dos bits menos significativos siempre en “00” (sólo instrucciones de 32 bits)
- Arquitectura load / store. Todas las operaciones aritmético lógicas son entre registros. Operaciones (lw/sw) especiales para leer y escribir memoria.

Ejemplo de instrucción (0x0e818193)



- `addi x3, x3, 232` // `x3 += 232`
 - Secuencia: S0, S1, S5, S2 (uso de la ALU combinacional).
 - En la figura se ve la simulación y las señales internas del procesador.

Escritura y habilitaciones por byte

Acceso alineado	addr[1:0]	wr[3:0]
SB en byte 0	00	0001
SB en byte 1	01	0010
SB en byte 2	10	0100
SB en byte 3	11	1000
SH en mitad baja / alta	00 / 10	0011 / 1100
SW	00	1111

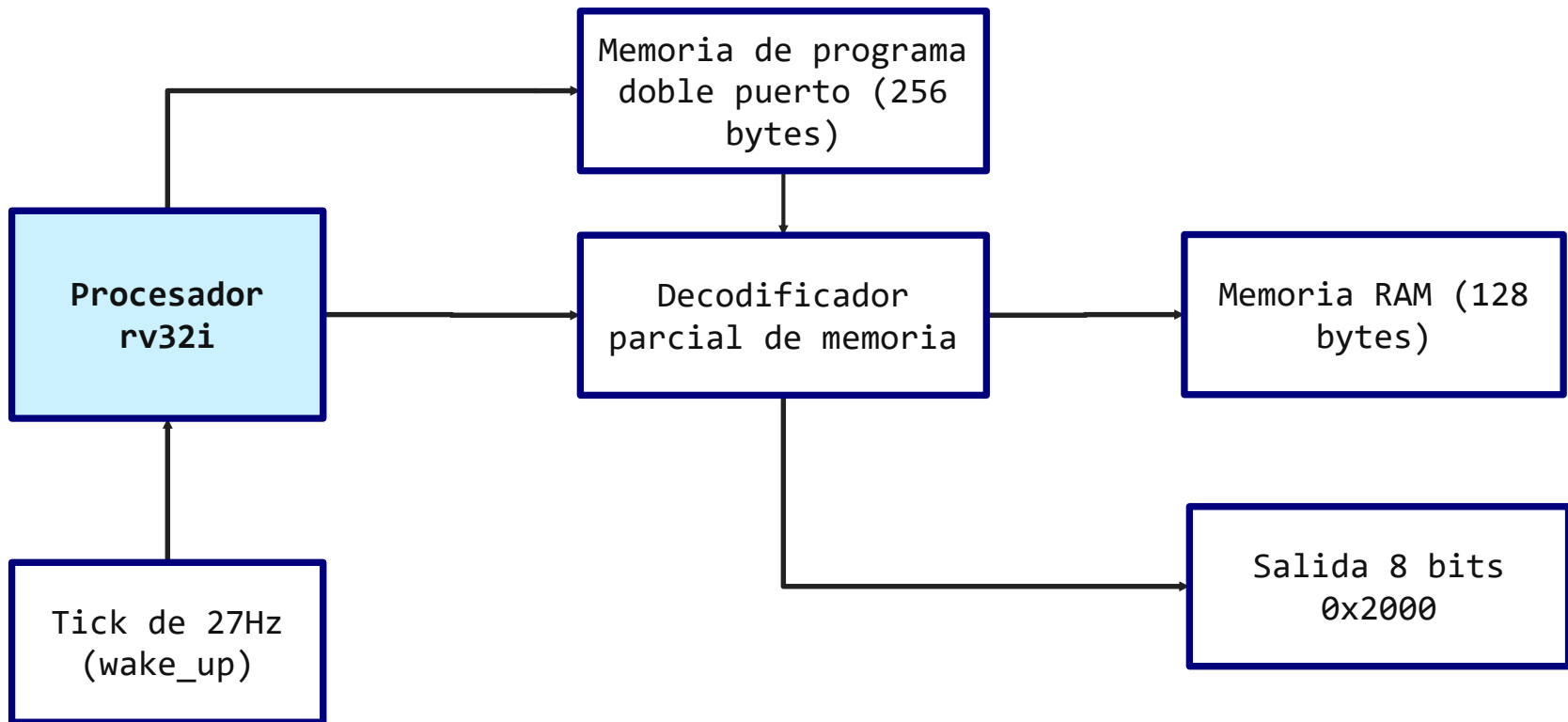
Salto, llamadas y retorno

Instrucción	Destino del PC	Dato para rd
Bxx	PC + inmediato B	Sin escritura de rd
JAL	PC + inmediato J	PC + 4
JALR	rs1 + inmediato I	PC + 4

- El opcode SYSTEM conduce a S6
- S6 mantiene el núcleo en reposo hasta wake_up = 1
- Con wake_up, ejecutar permite continuar con PC + 4
- El programa usa la palabra 0x10500073



Integración del sistema

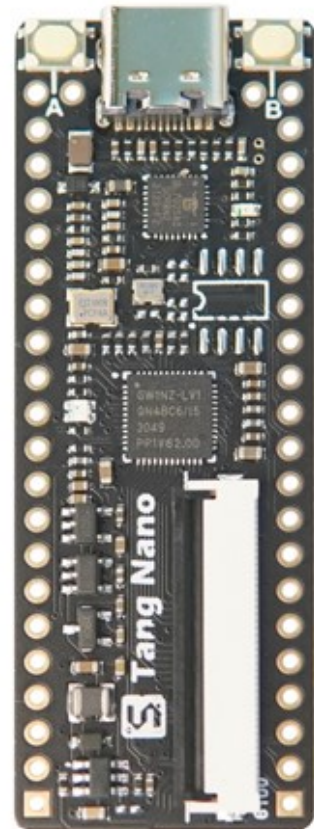


Memoria física y mapa lógico

Elemento	Dirección de uso	Capacidad real
RAM	0x00000000...0x0000007F	32 × 32 bits = 128 bytes
Puerto de salida	0x00002000	Registro de 8 bits
ROM por bus de datos	0x80000000...0x800000FF	64 × 32 bits = 256 bytes (visibles en el mapa de memoria)
ROM por PC		Segundo puerto para el bus de programa.

Reset y conexión de los LED

- Descripción funcional del hardware de prueba.
 - Cualquier pulsador resetea el sistema (activo bajo)
 - `rst_in <= key_a and key_b;`
- Se usa el led RGB activo en bajo
 - `nled <= NOT puerto_salida(2 downto 0);`
- Se usa la señal de reloj de 27 MHz de la placa.



De C a la ROM: archivos del sistema ejemplo

Etapa	Archivo / herramienta	Resultado
Programa	ejemplo.c	Patrón y espera
Arranque	start_juan.s	Pila, .data y .bss
Ubicación	micro_juan.ld	ROM y RAM
Construcción	Makefile + GCC	ELF y listado
Conversión	objcopy + generar_rom.py	ROM VHDL

Archivo del linker

Sección / símbolo	Dirección	Uso
.text y .rodata	0x80000000...	Código y constantes en ROM
.data	0x00000000	leds: variable en RAM inicializada con valor en ROM
.bss	0x00000004	Memoria RAM sin inicializar
_stack_ptr	0x00000080	Puntero a la pila (crece hacia direcciones menores)

Software de inicio

- De acuerdo al archivo `.ld` el código comienza en `_start`
- Antes de ejecutar `main()`:
 - Se envía el puntero a la pila (SP) al final de la memoria RAM.
 - Se copia el bloque `.rodata` (usando la etiqueta `_rdata`) que está luego de la memoria de programa a `_sdata`.
 - Toda la sección `.bss` se pone a cero
 - Se salta al símbolo `main`.

```
.section .text
.global _start
/*Pongo el Stack al fondo de la memoria y salto a main*/
_start:
    lui sp, %hi(_stack_ptr)
    addi sp, sp, %lo(_stack_ptr)

/*Variables Inicializadas*/
    lui x3, %hi(_rdata)
    addi x3, x3, %lo(_rdata)
    lui x4, %hi(_sdata)
    addi x4, x4, %lo(_sdata)
    lui x5, %hi(_edata)
    addi x5, x5, %lo(_edata)

InicLoop:
    beq x4, x5, Inicbss
    lw x6, 0(x3)
    sw x6, 0(x4)
    addi x3, x3, 4
    addi x4, x4, 4
    jal zero, InicLoop

Inicbss:
    lui x3, %hi(_sbss)
    addi x3, x3, %lo(_sbss)
    lui x4, %hi(_ebss)
    addi x4, x4, %lo(_ebss)

Loopbss:
    beq x3, x4, JumpMain
    sw x0, 0(x3)
    addi x3, x3, 4
    jal zero, Loopbss

JumpMain:
    jal zero, main
```

El programa de aplicación

```
#define PUERTO (0x2000)
#define __wfi() asm("wfi")
#define TICS_ESPERA (3)
#include <stdint.h>

uint32_t leds = 1;
uint32_t espera;
void desplaza_led(void)
{
    if(leds >= 0x80) leds = 1;
    else leds <<= 1;
}

// 01, 02, 04, 08, 10, 20,
// 40, 80, 01, 02, 04, ...

int main(void)
{
    volatile int *puerto_salida = (volatile int*)PUERTO;
    espera = TICS_ESPERA;
    while(1)
    {
        *puerto_salida = leds;
        if(!--espera)
        {
            espera = TICS_ESPERA;
            desplaza_led();
        }

        /*
         * Duermo el procesador hasta que se ponga en uno
         * el pin wake_up. Eso sucede periodicamente.
         * uso wfi o cualquier instrucción especial
         * (ecall, ebreak, etc).
         */
        __wfi();
    }
    return 0;
}
```

Compilar para el sistema implementado

- Se utilizó la el paquete estándar riscv64-unknown-elf
 - (`#apt install gcc-riscv64-unknown-elf` en Ubuntu)
- Se compila como:
 - `riscv64-unknown-elf-gcc`
 - `-march=rv32i` (arquitectura rv32i)
 - `-mabi=ilp32` (int, long, punteros de 32 bits)
 - `-ffreestanding` (sin sistema operativo)
 - `-O0` (sin optimizaciones - debug)
 - `-nostdlib` (sin biblioteca estándar)
 - `-nostartfiles` (sin archivos de arranque estándar)
 - `-Wl,` (opciones para el linker)
 - `-T,micro_juan.ld` (linker script)
 - `start_juan.s ejemplo.c` (archivos)
 - `-o ejemplo.out` (salida)

Generando la memoria de programa

- `objdump -d -M no-aliases ejemplo.out`
- `objcopy -O binary ejemplo.out ejemplo.bin`
- `python3 generar_rom.py ejemplo.bin`
- `riscv64-unknown-elf-size ejemplo.out`

□	text	data	bss	dec	hex	filename
□	232	4	4	240	f0	ejemplo.out

- Ejemplo:

- // Instrucción: `ADDI sp, x0, 128`
- // Palabra little-endian: `0x08000113`
- // Bytes: `13 01 00 08`

```
architecture Behavioral of programa is
    constant DEPTH : integer := 2**NBITS;
    type memoria_rom is array (0 to DEPTH-1) of
        std_logic_vector(31 downto 0);
    constant ROM : memoria_rom := (
        0 => x"08000113",
        1 => x"800001b7",
        2 => x"0e818193",
        3 => x"00000213",
        4 => x"00400293",
        5 => x"00520c63",
        6 => x"0001a303",
        7 => x"00622023",
```

Revisando memorias

Dato	Resultado adjunto	Lectura correcta
text	232 bytes	Código + constantes
data	4 bytes	Valor inicial de leds
bss	4 bytes	No ocupa bytes en ROM
Carga de ROM	236 bytes de 256 bytes	Quedan 20 bytes

Relación entre C y ensamblador

```
■ // C: if (leds >= 0x80) ...
■ 8000005c: 07f00793  addi a5, zero, 127
■ 80000060: 00e7f863  bgeu a5, a4, 80000070

■ // C: leds <= 1;
■ 80000074: 00179713  slli a4, a5, 1
```

```
void desplaza_led(void)
{
    if(leds >= 0x80) leds = 1;
    else leds <= 1;
}
```

```
80000048 <desplaza_led>:
80000048: ff010113      addi    sp,sp,-16
8000004c: 00112623      sw      ra,12(sp)
80000050: 00812423      sw      s0,8(sp)
80000054: 01010413      addi    s0,sp,16
80000058: 00002703      lw      a4,0(zero) # 0 <_start_ram>
8000005c: 07f00793      addi    a5,zero,127
80000060: 00e7f863      bgeu    a5,a4,80000070 <desplaza_led+0x28>
80000064: 00100713      addi    a4,zero,1
80000068: 00e02023      sw      a4,0(zero) # 0 <_start_ram>
8000006c: 0100006f      jal     zero,8000007c <desplaza_led+0x34>
80000070: 00002783      lw      a5,0(zero) # 0 <_start_ram>
80000074: 00179713      slli    a4,a5,0x1
80000078: 00e02023      sw      a4,0(zero) # 0 <_start_ram>
8000007c: 00000013      addi    zero,zero,0
80000080: 00c12083      lw      ra,12(sp)
80000084: 00812403      lw      s0,8(sp)
80000088: 01010113      addi    sp,sp,16
8000008c: 00008067      jalr    zero,0(ra)
```

La pila dentro de 128 bytes

Uso	Dirección / intervalo	Observación
Variable de patrón (led)	0x00...0x03	Inicializada a 1
Contador software (espera)	0x04...0x07	Arranque en 3 dentro de main
Stack frame desplaza_led()	0x50...0x5F	16 bytes
Stack frame main()	0x60...0x7F	32 bytes
SP inicial	0x80	Tope de la pila

Ocho bits de patrón y tres LED

Patrón	Pin activo en bajo	Efecto
0x01	nled[0], pin 9	Rojo (según pinout)
0x02	nled[1], pin 10	Azul
0x04	nled[2], pin 11	Verde (según pinout)
0x08...0x80	Ninguno de los tres	Todos apagados

Recursos de la implementación Gowin

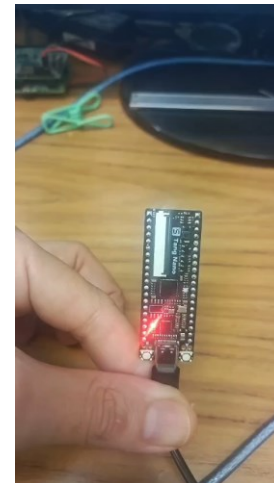
Recurso	Uso	Reporte
Logic	1017 / 1152	89 %
LUT	789	-
ALU	132	-
Registros	144 / 957	16 %
CLS	521 / 576	91 %
BSRAM	4 / 4	100 %
Puertos I/O	6 / 41	15 %

Análisis temporal estática (STA)

- Restricción: 27,000 MHz
- F_{MAX} reportada: 27,001 MHz
- Menor slack de setup: +0,002 ns
- El análisis reporta cero endpoints violados de setup y hold

Tiempos del sistema

- Reloj del sistema: 27 MHz.
- wake_up: un pulso cada 1 000 000 de ciclos ($\sim 37,04\text{ms}$)
- Cada patrón dura 3 ticks de wake_up ($\sim 111\text{ ms}$)

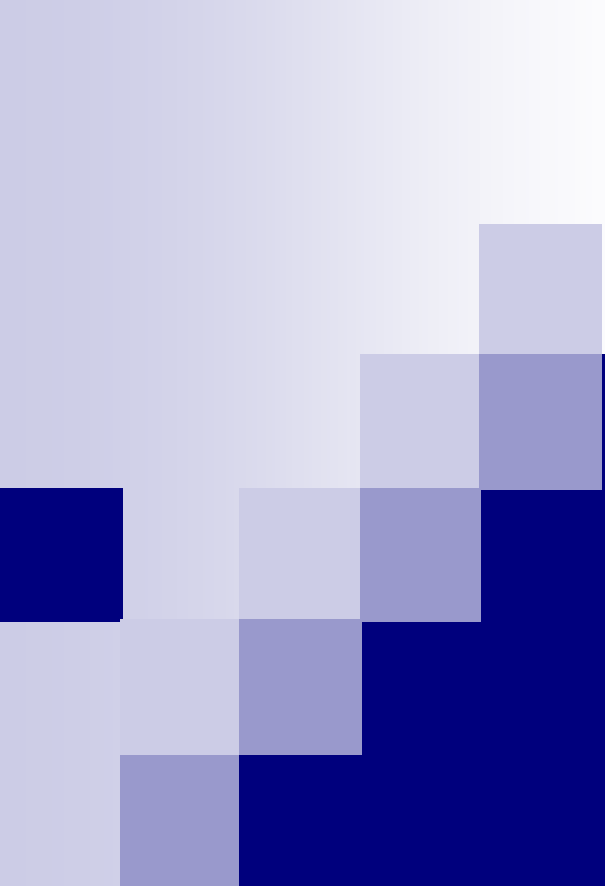


Conclusiones

- Se verificó que el ISA estándar permite usar herramientas reales.
- Se pudo analizar en detalle todo el proceso previo a la llamada al main()
- El circuito sigue siendo una implementación educativa que muestra un softcore en un dispositivo de bajo costo
- Se tomaron decisiones para simplificar el procesador, que inicialmente no afectan pero enseguida se vuelven necesarias:
 - SYSTEM no distingue WFI, ECALL, EBREAK y accesos CSR
 - Los saltos fuerzan los dos bits bajos del PC a cero
 - Los accesos de memoria desalineados no combinan palabras
 - No se implementa el modo máquina.
 - So se trata FENCE en la implementación.

Referencias

- Material de la presentación:
 - <https://github.com/esalarcon/riscv32i>
- Antecedentes:
 - <https://github.com/esalarcon/jfive>
- Referencias: especificación oficial RISC-V
 - <https://riscv.org/specifications/ratified/>



Muchas gracias